

AI Research OS

An Autonomous Research Assistant — Project Overview & Build Roadmap

A local-first AI system that continuously discovers, reads, understands, organizes, compares, and summarizes research papers while interacting naturally through voice or chat. Instead of answering questions about a single PDF, it manages an evolving knowledge base of research across multiple topics.

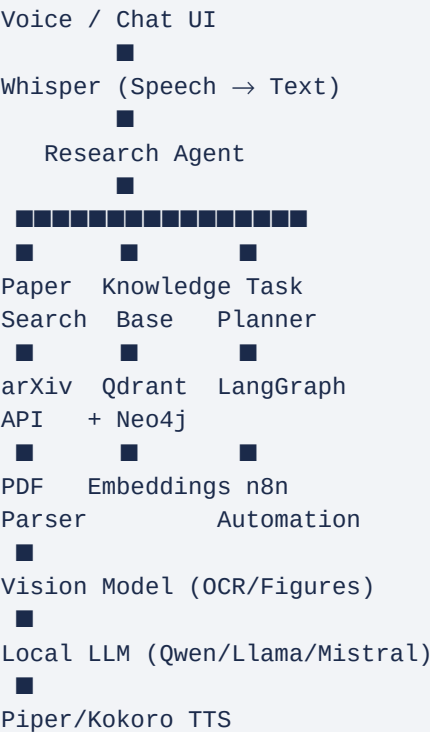
The Problem

Researchers spend hours on repetitive, manual work:

- Searching arXiv and related repositories
- Reading dozens of papers
- Taking notes
- Comparing methods
- Tracking new publications
- Writing literature reviews
- Organizing references

This system automates much of that workflow.

High-Level Architecture



Main Features

1. Automatic Paper Discovery

User says: "Find recent papers about GraphRAG published after January 2026." The agent searches arXiv, Semantic Scholar, Crossref, and ACL Anthology (for NLP), downloads papers automatically, stores metadata, extracts PDF text, and creates embeddings.

2. Intelligent Paper Understanding

Goes beyond chunking PDFs — extracts abstract, methodology, experiments, datasets, benchmarks, limitations, and future work, then stores it as structured information, e.g. a JSON record with title, task, dataset, backbone model, accuracy, and limitations.

3. Multi-Paper Comparison

Example prompt: "Compare all GraphRAG papers using HotpotQA." Produces a side-by-side table (dataset, model, accuracy, speed) with the LLM explaining the differences.

4. Literature Review Generator

Instead of summarizing one paper, generates a structured draft: Introduction, Background, Existing Methods, Limitations, Research Gaps, Future Directions, References — essentially a first draft of a survey paper.

5. Knowledge Graph

Replaces flat RAG with a graph structure (Neo4j) linking Paper → Authors → Concepts → Datasets → Methods → Citations, enabling queries like "Which papers use Qwen and evaluate on MMLU?"

6. Figure Understanding

Uses vision models (Qwen2.5-VL, InternVL, Gemma Vision) to extract and explain architecture diagrams, flowcharts, tables, and graphs from papers.

7. Voice Research Mode

Natural spoken conversation, e.g. "Explain this paper like I'm a beginner," followed by "Why did they choose LoRA instead of full fine-tuning?" — answered via TTS.

8. Weekly Research Automation (n8n)

Each morning: search arXiv → filter topics → download PDFs → process → embed → generate summary → send via email/Notion → voice notification. No manual searching required.

9. Citation Assistant

Example: "Give me BibTeX for all papers discussing GraphRAG." Automatically exports BibTeX, APA, IEEE, and MLA formats.

10. Experimental Comparison

Extracts GPU, parameter count, training time, memory, and FLOPs across papers, then generates comparison charts.

11. Research Timeline

Visualizes the evolution of a field over time, e.g. Dense Retrieval (2022) → Hybrid RAG (2023) → GraphRAG (2024) → Agentic RAG (2025) → Memory RAG (2026).

12. Ask Across Hundreds of Papers

Example: "Which methods outperform ColBERT on long-context QA?" Searches every paper, reasons across them, and returns an answer with citations.

13. Dataset Explorer

Automatically finds and groups datasets, metrics, benchmarks, and leaderboards across the corpus.

14. Research Gap Finder

Example: "What research gaps still exist in GraphRAG?" The LLM analyzes limitations, future work, and open problems across papers to produce possible project ideas.

15. Personal Research Notebook

Every interaction is summarized, stored, indexed, and linked, so the system can later answer "What did I think about GraphRAG last month?"

AI Agents (LangGraph)

Rather than one monolithic LLM, the system uses specialized agents, each with a single responsibility:

- Planner Agent
- Search Agent
- Reading Agent
- Vision Agent
- Comparison Agent
- Writing Agent
- Citation Agent
- Memory Agent

Local LLM Stack

Component	Options
LLM	Qwen3 8B, Llama 3.2, Mistral, Gemma
Speech-to-Text	Whisper.cpp

Text-to-Speech	Piper, Kokoro
Embeddings	BAAI BGE, Jina, E5
Vision	Qwen2.5-VL

Where n8n Fits

n8n isn't just an add-on — it's the automation backbone. Workflows can include:

- Daily arXiv monitoring for selected topics
- New GitHub repository releases related to AI frameworks
- Google Scholar alerts (replace with Semantic Scholar/OpenAlex API — Scholar has no public API and scraping violates its ToS)
- Slack/Discord notifications
- Automatic Notion page creation for each processed paper
- Calendar reminders for conferences or submission deadlines
- Email digests of newly discovered research

Fine-Tuning (LoRA/QLoRA)

Rather than fine-tuning a general-purpose LLM, target one specific capability. **Recommended: Structured Information Extraction** — fine-tune the model to convert research papers into structured JSON (datasets, models, metrics, limitations, future work), improving consistency over prompt engineering alone, and giving you a clean before/after accuracy comparison as an interview artifact.

Other options considered (lower priority, stretch only):

- Research paper summarization — train on arXiv/PubMed summarization datasets
- Scientific question answering — fine-tune on datasets such as SciQ
- Citation recommendation — lightweight model suggesting citations from a paragraph or topic

Tech Stack

Layer	Technology
Frontend	Next.js or React
Backend	FastAPI
Agent Framework	LangGraph
Workflow Automation	n8n
Local LLM	Ollama or vLLM
Speech-to-Text	Whisper.cpp

Text-to-Speech	Piper or Kokoro
Vision	Qwen2.5-VL
Vector Database	Qdrant
Graph Database	Neo4j (introduced in Phase 2)
Cache	Redis
Object Storage	MinIO
Embeddings	BGE-M3 or Jina Embeddings
Fine-Tuning	Unsloth + PEFT (QLoRA)
Evaluation	Ragas + DeepEval
Monitoring	Langfuse
Deployment	Docker Compose

Build Roadmap (2 Phases)

Since this is now the sole active project, the roadmap is intentionally kept to two phases so there is always a complete, demoable, interview-ready product — rather than a sprawling, half-finished system.

Phase 1 — Core Research Assistant

The full foundation: everything needed to make the system genuinely useful end-to-end, before it needs to be "smart."

Ingestion & Storage

- arXiv API search + PDF download
- PDF parser → structured extraction (abstract, methodology, experiments, datasets, benchmarks, limitations, future work) as JSON
- Embeddings (BGE-M3 or Jina) → Qdrant
- SQLite/Postgres for metadata (Neo4j deferred to Phase 2)

Core Agent Loop (LangGraph)

- Planner → Search Agent → Reading Agent → Writing Agent (4 agents; Comparison/Citation/Memory merged into Writing Agent for now)
- Local LLM via Ollama — Qwen3 8B as primary model, others swappable

User-Facing Features

- Automatic paper discovery (arXiv only — Semantic Scholar/Crossref/ACL added later)
- Intelligent paper understanding (structured extraction)
- Multi-paper comparison (table output)
- Literature review generator (shares a pipeline with comparison — build together)
- Ask-across-papers Q&A with citations back to source paragraphs (critical for trust — do not skip)
- Citation assistant (BibTeX/APA/IEEE export — cheap to add once metadata exists)

Interface

- Chat UI only (Next.js/React + FastAPI) — no voice yet
- No n8n yet — a manual "search now" trigger is sufficient

This alone is a complete, demoable, interview-ready product. Ship this before starting Phase 2.

Phase 2 — Intelligence & Automation Layer

Once Phase 1 works reliably, layer on the differentiators that separate this from "yet another RAG chatbot."

Depth Features

- Knowledge graph (Neo4j) — now justified, since comparison and gap-finding queries need graph traversal
- Research gap finder (built on the graph plus limitations/future-work fields already extracted)
- Research timeline visualization
- Dataset/benchmark explorer
- Experimental comparison charts (GPU, params, training time, FLOPs)
- Figure/vision understanding (Qwen2.5-VL) — expensive to build well, so it comes last among the "smart" features

Automation & Polish

- n8n weekly automation (arXiv monitoring → process → email/Notion/Slack digest)
- Voice mode (Whisper.cpp + Piper/Kokoro) — strong demo polish, lower functional value vs. effort, so treat as optional/stretch even within Phase 2
- Personal research notebook / memory across sessions
- Fine-tuning: structured extraction model (Unsloth + QLoRA) with a before/after eval
- Langfuse monitoring, Docker Compose packaging for the full stack

Why This Split

Phase 1 forces the hard trust problem to be solved early — citation traceability and extraction accuracy — rather than bolting it on after building flashy graph or vision features. Phase 2 is where the project differentiates itself, but only after Phase 1 proves the core loop actually works.

Open Items to Address Along the Way

- Extraction traceability: every extracted field (e.g. "accuracy: 91.2") should link back to the source page/paragraph so comparison tables are verifiable, not just plausible
- Data source legality: avoid scraping Google Scholar (no public API, against ToS) — use Semantic Scholar or OpenAlex instead
- Licensing: note how non-open-access papers are handled (e.g. abstract-only processing) versus fully open arXiv PDFs